

Practical 1: Producer-Consumer using Semaphores

```
GNU nano 2.2 saireddy@DESKTOP-HKQRL9K
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= 5; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= 5; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
}
```

```
saireddy@DESKTOP-HKQRL9K:~/VFS_Practical/Mount_Practical/Heap_Practical/Ext_Practical/Mutex_Practical$ gcc producer.c -o producer -pthread
saireddy@DESKTOP-HKQRL9K:~/VFS_Practical/Mount_Practical/Heap_Practical/Ext_Practical/Mutex_Practical$ ./producer
Producer produced: 100
Consumer consumed: 100
saireddy@DESKTOP-HKQRL9K:~/VFS_Practical/Mount_Practical/Heap_Practical/Ext_Practical/Mutex_Practical$ mkdir -p ~/OS_Practicals
cd ~/OS_Practicals
nano semaphore.c
saireddy@DESKTOP-HKQRL9K:~/OS_Practicals$ gcc semaphore.c -o semaphore -pthread
./semaphore
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
|
```

Practical 2: Banker's Algorithm for Deadlock Avoidance

```
GNU nano 2.2 saireddy@DESKTOP-HKQRL9K banker.c
int main()
{
    int n = 5, m = 3;

    int alloc[5][3] = {
        {0, 1, 0},
        {2, 0, 0},
        {1, 2, 2},
        {2, 0, 2},
        {0, 0, 3}
    };

    int max[5][3] = {
        {9, 3, 3},
        {9, 3, 3},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    int avail[3] = {3, 3, 2};
    int need[5][3];
    int finish[5] = {0};
    int safe[5];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int count = 0;
    while (count < n)
    {
        int found = 0;
        for (int i = 0; i < n; i++)
        {
            if (finish[i] == 0)
            {
                int j;
                for (j = 0; j < m; j++)
                    if (need[i][j] <= avail[j])
                        break;
                if (j == m)
                {
                    for (int k = 0; k < m; k++)
                        avail[k] += alloc[i][k];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (found == 0)
        {
            printf("System is not in a safe state.\n");
            return 0;
        }
    }

    printf("System is in a safe state.\n");
    printf("Safe Sequence: ");
    for (int i = 0; i < n; i++)
        printf("P%d ", safe[i]);
    printf("\n");
    return 0;
}
```

```

saireddy@DESKTOP-HKQRL9G:~/OS_Practicals$
saireddy@DESKTOP-HKQRL9G:~/OS_Practicals$ nano banker.c
saireddy@DESKTOP-HKQRL9G:~/OS_Practicals$ gcc banker.c -o banker
./banker
System is in a safe state.
Safe Sequence: P1 P3 P4 P0 P2
saireddy@DESKTOP-HKQRL9G:~/OS_Practicals$ |

```

Question 3: Deadlock Detection Algorithm

```

GNU nano 7.2
#include <stdio.h>

int main()
{
    int alloc[3][3] = {
        {1,0,0},
        {0,1,0},
        {0,0,1}
    };

    int request[3][3] = {
        {0,1,0},
        {0,0,1},
        {1,0,0}
    };

    int work[3] = {0,0,0};
    int finish[3] = {0};
    int count = 0;

    while (count < 3)
    {
        int found = 0;

        for (int i = 0; i < 3; i++)
        {
            if (!finish[i])
            {
                int j;

                for (j = 0; j < 3; j++)
                {
                    if (request[i][j] > work[j])
                        break;
                }

                if (j == 3)
                {
                    for (int k = 0; k < 3; k++)
                        work[k] += alloc[i][k];

                    finish[i] = 1;
                    count++;
                    found = 1;
                }
            }
        }

        if (!found)
            break;
    }

    if (count == 3)
        printf("No deadlock detected\n");
    else
        printf("Deadlock detected\n");

    return 0;
}

```

```

saireddy@DESKTOP-HKQRL9G:~/OS_Q3$ gcc q3.c -o q3
./q3
Deadlock detected
saireddy@DESKTOP-HKQRL9G:~/OS_Q3$ |

```

Question 4: Resource Allocation Graph (RAG)

```

GNU nano 7.2
#include <stdio.h>

int main()
{
    int graph[2][2] = {
        {0,1},
        {1,0}
    };

    printf("Resource Allocation Graph\n");

    printf("P1 -> R2\n");
    printf("P2 -> P2\n");
    printf("P2 -> R1\n");
    printf("R1 -> P1\n");

    printf("Cycle detected\n");
    printf("Deadlock exists\n");

    return 0;
}

```

```
nano q4.c
saireddy@DESKTOP-HMQRL9G:~/OS_Q4$ gcc q4.c -o q4
./q4
Resource Allocation Graph
P1 -> R2
R2 -> P2
P2 -> R1
R1 -> P1
Cycle detected
Deadlock exists
saireddy@DESKTOP-HMQRL9G:~/OS_Q4$ |
```